Universidad de SanAndrés

Ingeniería de Software y Datos
MIA 203

PRIMEROS PASOS

¿Qué es un contenedor?¿Qué es una imagen?¿Qué es…?

A **container** is a runnable instance of an image. You can start, stop, move, or delete a container. Containers are isolated from one another and the host system but share the OS kernel. They provide a lightweight and consistent way to run applications.

An **image** is a read-only template used to create containers. It typically includes a base operating system and application code packaged together using a **Dockerfile**. Images are versioned using tags and can be pushed to or pulled from a container registry like Docker Hub.

In an image, a **layer** is a modification represented by an instruction in the Dockerfile. Layers are applied in sequence to the base image to create the final image. Unchanged layers are cached, making image builds faster and more efficient.

A **base image** is an image you designate in a FROM directive in a Dockerfile. It defines the starting point for your build. Dockerfile instructions create additional layers on top of the base image. A Dockerfile with the FROM scratch directive uses an empty base image.

Build is the process of building Docker images using a Dockerfile. The build uses a Dockerfile and a "context". The context is the set of files in the directory in which the image is built.

https://docs.docker.com/reference/glossary/

¿Todo en orden?

Correr el siguiente comando:
`docker --version`
Y luego:
`docker info`

En Linux, (si el usuario no está en el grupo docker) anteceder sudo a los comandos. En Windows/Mac, verificar que la aplicación Docker Desktop esté abierta en segundo plano.

¿Cómo empezamos?

Inspeccionemos nuestros primeros contenedores.
Ejecutar:
`docker run -d -p 8080:80 docker/welcome-to-docker`
Luego, abrir un navegador e ingresar a http://localhost:8080/

¿Qué hicimos exactamente?
docker run
Es el comando principal que le indica a Docker que cree y ejecute un nuevo **contenedor** a partir de una **imagen**.
Si la imagen no está descargada, Docker la buscará automáticamente en el registro (Docker Hub) y la descargará antes de ejecutarla.
-d (o --detach): El contenedor se ejecuta en segundo plano permitiendo seguir usando otros comandos en la terminal inmediatamente.
-p 8080:80 (o --publish): Define el mapeo de puertos entre nuestra computadora (host) y el contenedor. Sigue la estructura PuertoHost:PuertoContenedor:

- **8080** (Host): El puerto que se abrirá en nuestra máquina local.
- **80** (Contenedor): El puerto interno en el que la aplicación del contenedor está escuchando el tráfico (usualmente el puerto web estándar).
- Cualquier tráfico que enviemos a http://localhost:8080 en el navegador se redirigirá automáticamente al puerto 80 dentro del contenedor.

docker/welcome-to-docker: Es el nombre de la imagen que se va a utilizar para crear el contenedor. En este caso, es una imagen oficial de Docker que contiene una página web de bienvenida.

¿Qué está pasando?

Ejecutar y analizar el resultado de los siguientes comandos:
`docker images`
`docker ps`
El contenedor **docker/welcome-to-docker** va a seguir corriendo hasta que lo detengamos. Para pararlo usamos el comando **docker stop** con el id correspondiente (que podemos ver en docker ps).
Ejecutar:
`docker stop` *`<el id>`*
Volver a correr
`docker ps`
Luego,
`docker ps -a`
¿Qué cambió?

¿Cuántos contenedores puedo hacer con una imagen?

```
docker run -d --name prueba1 nginx
docker run -d --name prueba2 nginx
```

`--name prueba1`: Le asigna el nombre "prueba1" al contenedor. Sin ese parámetro, Docker le asigna un nombre aleatorio. Tener un nombre fijo nos simplifica el uso de

comandos como stop o rm (borrado).
nginx: Es la imagen base que va a utilizar. Docker buscará la imagen oficial de Nginx en nuestra computadora; si no la encuentra, la descargará automáticamente desde el registro de Docker Hub.
¿Qué pasa cuando se ejecuta? El contenedor comenzará a funcionar de inmediato e iniciará el servidor web. Sin embargo, como el comando no incluye una redirección de puertos (el parámetro -p), el servidor Nginx estará activo internamente pero no podremos acceder a él desde el navegador.

docker ps
docker logs *<el contenedor del que queremos ver los logs, podemos poner prueba1, prueba2 o sus IDs>*
¿Son dos contenedores diferentes?
¿Utilizan la misma imagen?
¿Pueden existir simultáneamente aunque ejecuten el mismo programa?
¿Qué va a pasar si eliminamos prueba1? ¡Probemos!
docker stop prueba1
docker rm prueba1
docker ps
docker ps -a
Detener y borrar todos los contenedores creados utilizando stop, rm y su nombre/id.

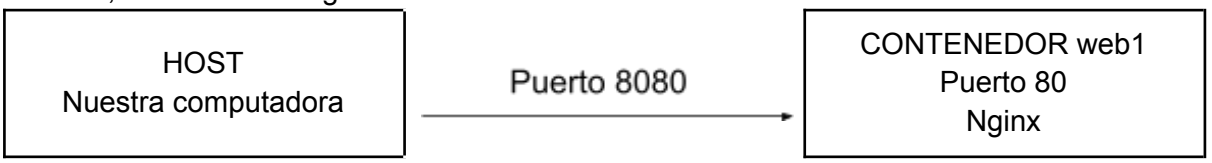
¿Qué pasa con los puertos?

Ejecutar:
docker run -d --name web1 -p 8080:80 nginx
Ver el resultado en el navegador ingresando en http://localhost:8080
Intentar ejecutar el siguiente comando y ver qué ocurre
docker run -d --name web2 -p 8080:80 nginx
El segundo contenedor no puede utilizar el puerto 8080 del host porque ya está ocupado.
¿Cuál es el puerto del host?
¿Cuál es el puerto dentro del contenedor?
¿Por qué dos contenedores pueden escuchar ambos en el puerto 80?
¿Por qué no pueden ambos publicar 8080?
Detenerlo, borrarlo y probar:
docker run -d --name web2 -p 8081:80 nginx
docker port web1
docker port web2

Cuidado con localhost

Cuando hablamos de localhost estamos haciendo referencia al mismo equipo/entorno desde donde estamos haciendo la conexión.
Por ejemplo, http://localhost:8081 significa, conceptualmente "Conectate al puerto 8081 de esta misma computadora."
Pero dentro del contenedor web **localhost** se refiere al propio contenedor web.

Es decir, tenemos dos lugares diferentes:



Nginx está escuchando en el puerto 80 dentro del contenedor.
Nuestra computadora puede utilizar el puerto 8080.
Por eso necesitamos decirle a Docker que conecte ambos con la sintaxis
`-p puerto_host:puerto_contenedor`
"Las conexiones que lleguen al puerto 8080 del host deben ser dirigidas al puerto 80 del contenedor."
Esto es publicar un puerto. El contenedor está aislado del host. Por eso no basta con que Nginx esté ejecutándose dentro del contenedor. Necesitamos crear una forma de acceso desde el host.
Ahora podemos abrir http://localhost:8080 en el navegador.

Antes de continuar, borramos forzosamente ambos contenedores:
`rm -f web1 web2`

Para cada contenedor solo existen los archivos de su imagen. Además, pueden tener archivos creados durante su ejecución y archivos provenientes de volúmenes o bind mounts.
Se comunica con su propia red, tiene su IP y sus puertos. Por eso en nuestra prueba podían utilizar ambos el puerto 80.
Los contenedores comparten los recursos del host y Docker nos permite establecer límites de CPU y memoria.

¿Y la imagen?

Una vez creada una imagen, no se puede modificar. Solo se puede crear una nueva imagen o añadirle cambios.
Las imágenes están compuestas por capas. Cada capa representa un conjunto de cambios en el sistema de archivos que añaden, eliminan o modifican archivos.
Estos dos principios permiten extender o añadir funcionalidades a las imágenes existentes. Por ejemplo, para desarrollar una aplicación en Python, podemos partir de la imagen de Python y añadir capas adicionales para instalar las dependencias de nuestra aplicación e incorporar nuestro código. Si varias imágenes comparten las mismas capas base, Docker las almacena una sola vez .
Por ejemplo, la **base** python:3.13-slim incluye archivos de un Linux ligero, librerías del sistema y Python instalado. Si le agregamos nuestro **Dockerfile** con dependencias, el código y los comandos necesarios obtendremos una **imagen** para correr nuestra aplicación.

Podemos listar las imágenes que tenemos instaladas con
`docker image ls`

¿Cómo creamos un Dockerfile?

Crear una carpeta llamada mi-docker-app y entrar en ella desde la terminal o VS Code.
Crear un archivo de texto llamado `requirements.txt` con el siguiente contenido:
`redis==5.0.3`
Crear un archivo llamado `Dockerfile` (sin extensión) con las siguientes instrucciones:

```
# 1. Partimos de una imagen base oficial de Python ligera
FROM python:3.13-slim
# 2. Definimos la carpeta de trabajo dentro del contenedor
WORKDIR /app
# 3. Copiamos e instalamos dependencias primero (Aprovecha la Caché)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# 4. Copiamos el resto de nuestros archivos al contenedor
COPY script.py .
# 5. Indicamos el comando que debe ejecutarse al iniciar
CMD ["python", "script.py"]
```

Las instrucciones de construcción generan capas y metadatos que Docker puede reutilizar durante el build. La forma exacta en que se materializan depende del constructor y de la instrucción.
Capa base: La primera instrucción (FROM) establece la base, usualmente un sistema operativo minimalista como Alpine o Ubuntu.
Solo lectura: Las capas de la imagen no se pueden modificar una vez creadas. Si cambiamos un archivo, Docker añade una nueva capa con la actualización en lugar de reescribir la anterior.
https://docs.docker.com/build/building/best-practices/

¿Y para construir la imagen?

Vamos a construir una imagen llamada mi-script a partir del Dockerfile creado.
`docker build -t mi-script:v1 .`

`docker build`: Es el comando base de Docker que le indica al sistema que debe iniciar el proceso de construcción (build) de una nueva imagen.
`-t mi-script:v1`: -t (o --tag) sirve para darle un nombre (y opcionalmente una etiqueta o versión) a la imagen que estamos creando. En este caso, la imagen se llamará mi-script y v1 será su versión. Si no especificamos una versión, Docker le asignará automáticamente la etiqueta :latest
. (el punto final): Indica el contexto de construcción, es decir, la ruta del directorio donde Docker debe buscar el archivo Dockerfile y los archivos del proyecto. El punto significa "este directorio actual" o "la carpeta en la que estoy parado en la terminal"

Ahora ya podemos crear un contenedor a partir de la imagen:
`docker run mi-script:v1`

y verificar que la imagen forma parte de las imágenes de nuestro equipo:
docker image ls

¿Cómo era eso de la caché?

Modificar el archivo script.py, volver a construir la imagen (ejecutar el comando build igual que antes) y analizar el resultado en la terminal.
Docker no volverá a instalar las bibliotecas porque requirements.txt no cambió, en los pasos que no se modifican vamos a poder leer “Using cache”.

¿De dónde salen los datos?

Por defecto, los archivos creados dentro de un contenedor desaparecen cuando el contenedor se borra.
Un **Bind Mount** vincula una carpeta de nuestra PC directamente con una carpeta dentro del contenedor.
Vamos a ejecutar el contenedor vinculando nuestro directorio actual con la carpeta /app del contenedor:
En Windows:
`docker run -v ${PWD}:/app mi-script:v1`
En otros S.O.:
`docker run -v $(pwd):/app mi-script:v1`
Luego, vamos a modificar lo que imprime el archivo script.py y volver a correr el mismo comando sin volver a construir la imagen. El contenedor leerá la versión actualizada del archivo.
Los **volúmenes nombrados** son espacios de almacenamiento creados y gestionados enteramente por Docker, útiles por ejemplo para bases de datos.
Creamos un volumen:
`docker volume create mis_datos`
`docker volume ls`
Luego, vamos a correr el siguiente comando:
`docker run --rm -v mis_datos:/datos alpine sh -c "echo 'Guardado en el volumen' > /datos/mensaje.txt"`

`--rm`: Indica que el contenedor debe eliminarse automáticamente en cuanto termine de ejecutarse el comando. Esto es ideal para tareas rápidas o de mantenimiento, ya que evita acumular contenedores basura.
`-v mis_datos:/datos`: Monta un volumen de Docker. Vincula un volumen llamado mis_datos (si no existe, Docker lo crea automáticamente) con la carpeta /datos que está dentro del contenedor. Todo lo que ocurra en esa carpeta se guardará fuera del contenedor.
`alpine`: Es la imagen base que se va a descargar y utilizar. Alpine Linux es una distribución extremadamente ligera, por lo que arranca casi instantáneamente y es perfecta para este tipo de tareas.
`sh -c "..."`: Le ordena al contenedor que abra una terminal tipo Shell (sh) y ejecute el

comando entre comillas (-c significa command).
`"echo 'Guardado en el volumen' > /datos/mensaje.txt"`: Es la acción final. Escribe el texto "Guardado en el volumen" y lo guarda en un archivo nuevo llamado mensaje.txt dentro de la carpeta /datos.

A continuación vamos a ejecutar:
`docker run --rm -v mis_datos:/datos alpine cat /datos/mensaje.txt`
Este comando sirve exactamente para leer el contenido del archivo que creamos en el paso anterior.

`docker run --rm`: Al igual que antes, crea un contenedor temporal y lo elimina automáticamente en cuanto termina de leer el archivo.
`-v mis_datos:/datos`: Conecta (monta) el mismo volumen virtual llamado mis_datos en la carpeta /datos de este nuevo contenedor. Gracias a esto, el contenedor tiene acceso a los archivos guardados previamente.
`alpine`: Vuelve a usar la imagen ligera de Alpine Linux para que el proceso sea rápido.
`cat /datos/mensaje.tx`t: Es el comando de Linux que se ejecuta dentro del contenedor. **`cat`** sirve para mostrar en la pantalla (terminal) el contenido del archivo mensaje.txt.

Diferencia entre Bind Mount y los volúmenes de Docker: Los volúmenes son administrados por Docker en una zona aislada del sistema, mientras que los bind mounts dependen de la estructura de carpetas específica de nuestro equipo local (host).

Para borrar un volumen:
`docker volume rm` *<nombre del volumen que puede obtenerse con el comando ls>*
Si el volumen está en uso en algún contenedor no se va a poder borrar, incluso si el contenedor no está corriendo. Vamos a tener que borrarlo primero.
A la hora de trabajar con datos pesados (como pesos de modelos) no va a ser conveniente incluirlos en el Dockerfile. Siempre conviene utilizar bind o volúmenes y montarlos en tiempo de ejecución.

¿Y si quiero usar más de un contenedor?

Cuando una aplicación necesita más de un servicio (por ejemplo, código en Python y una Base de Datos) ¿Instalamos todo en un solo contenedor? ¿Ejecutamos varios contenedores? Una buena práctica para los contenedores es que cada uno realice una sola función y la realice correctamente. Si bien existen excepciones a esta regla, conviene evitar que un mismo contenedor realice múltiples tareas.
¿Cómo configuramos y conectamos varios contenedores sin tener que ejecutar muchos comandos `docker run` manualmente?
Docker Compose permite definir varios servicios, sus imágenes, puertos, variables de entorno, volúmenes y redes en un único archivo YAML. Si incluímos este archivo en

nuestro repositorio de código, cualquiera que lo clone podrá empezar a usarlo con un solo comando.
Dockerfile versus Compose
Dockerfile: describe cómo construir una imagen.
Compose: describe cómo ejecutar y conectar varios servicios.

¿Cómo hacemos nuestro primer Compose?

Compose utiliza archivos escritos en YAML, un formato de texto que representa información estructurada mediante indentación.
En la misma carpeta donde tenemos nuestro Dockerfile crear un archivo llamado **compose.yaml**
En el, escribir:

```
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"

  db:
    image: postgres:18
    environment:
      POSTGRES_PASSWORD: “1234”
      POSTGRES_USER: example
      POSTGRES_DB: exampledb
```

`services:`
Es la etiqueta raíz que le indica a Docker Compose que abajo comenzará la lista de los contenedores (servicios) que componen nuestra aplicación.
https://docs.docker.com/reference/compose-file/services/

`web:`
Es el nombre interno que le damos a nuestro primer contenedor. Podemos usarlo para que otros contenedores se comuniquen con él.
`image: nginx:latest`
Indica que este contenedor se descargará y creará a partir de la imagen oficial de Nginx en su versión más reciente (latest).
`ports: - "8080:80"`
Mapea los puertos del contenedor con los de nuestra computadora (host).
Si entramos en el navegador a http://localhost:8080, accederemos al servidor Nginx.

El servicio de base de datos:
`db:`
Es el nombre del segundo contenedor (la base de datos).

image: postgres:18
Le dice a Docker que descargue y use una imagen oficial y ya lista de PostgreSQL versión 18 desde el repositorio público (Docker Hub).
environment:
Inicia la sección para definir variables de entorno, que sirven para configurar aspectos internos del contenedor cuando se inicia.
POSTGRES_PASSWORD: "1234"
Establece la contraseña obligatoria para el usuario administrador por defecto (postgres). En este caso, la clave de acceso a la base de datos será 1234. **Nota:** para información sensible, como contraseñas o claves, no conviene escribir directamente en un archivo Compose que se vaya a subir a un repositorio. En esta práctica solo lo utilizamos a modo de ejemplo.
POSTGRES_USER: example
Crea un nuevo usuario adicional con privilegios de superusuario llamado example.
POSTGRES_DB: exampledb
Crea automáticamente una base de datos predeterminada llamada exampledb al iniciar el contenedor por primera vez.

Tenemos ahora dos contenedores. Podemos iniciar ambos servicios con:
docker compose up
Observar qué ocurre.
Luego, en otra terminal (en la misma carpeta), ejecutar:
docker compose ps
docker ps
¿Cuántos contenedores se crearon?
¿Qué imagen utiliza cada uno?
docker compose up crea, enciende y conecta todos los contenedores definidos en nuestro archivo. Si agregamos el parámetro -d, se ejecutará en segundo plano (background).
docker compose stop detiene los contenedores, pero no los elimina.
Luego podemos volver a iniciarlos con **docker compose start**
docker compose rm Elimina contenedores detenidos del proyecto.
docker compose down: Apaga y elimina los contenedores, redes y volúmenes temporales creados por up. Los volúmenes nombrados **no se eliminan por defecto**.

Variables de entorno en Compose

Muchas aplicaciones necesitan configuraciones que puede cambiar sin modificar la imagen o datos que queremos resguardar como contraseñas.
Podemos crear un archivo llamado .env en la misma carpeta. Dentro del archivo ponemos los valores que queremos resguardar o configurar como variables de entorno: DB_PASSWORD=mi_pass.
En nuestro archivo de Compose la llamaremos usando la sintaxis ${DB_PASSWORD}. Docker Compose la reemplazará automáticamente al encender el servicio.

¿Qué significa depends_on?

Si en nuestro archivo .yaml agregamos en el servicio de web

```
depends_on:
      - db
```

Es útil para indicar el orden de inicio:
primero db, luego web

Que un contenedor haya comenzado no significa necesariamente que el servicio que contiene ya esté listo para recibir conexiones. Por ejemplo, el contenedor de PostgreSQL puede estar ejecutándose mientras PostgreSQL todavía está terminando de inicializarse. Para situaciones en las que necesitamos comprobar que un servicio realmente está disponible podemos utilizar un healthcheck.

Healthcheck

El healthcheck le permite a Docker comprobar periódicamente si el servicio está funcionando correctamente.
Para utilizarlo, debemos modificar nuestro yaml una vez más:

```
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:18
    environment:
      POSTGRES_PASSWORD: "1234"
      POSTGRES_USER: example
      POSTGRES_DB: exampledb
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U example -d exampledb"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s
```

healthcheck: Utiliza la herramienta nativa de PostgreSQL pg_isready para verificar la conectividad de la base de datos usando el usuario y la base de datos que definimos.

`interval, timeout, retries:` Configuran los tiempos de chequeo. Intentará cada 10 segundos y esperará hasta 5 reintentos antes de marcarlo como fallido.
`start_period:` Le da 10 segundos de gracia a PostgreSQL para arrancar antes de empezar a contar los fallos del healthcheck.
`depends_on:` Asegura que el contenedor web no se inicie hasta que el estado del healthcheck de db pase a estar en verde (service_healthy).

¿Cómo se conectan los servicios?

Cuando Compose crea una aplicación con varios servicios, crea una red para ellos.
Por defecto, los servicios de un mismo proyecto Compose pueden comunicarse utilizando el nombre del servicio.
En nuestro ejemplo tenemos los servicios web y db.
Desde el contenedor web, el nombre db identifica al servicio de PostgreSQL dentro de la red de Compose. Esto significa que una aplicación dentro de web no necesita conocer necesariamente la dirección IP de db. Puede utilizar db como nombre del servidor.
Desde nuestra computadora utilizamos un puerto publicado, pero entre contenedores utilizamos la red interna por eso no necesitamos publicar el puerto de PostgreSQL en el host para que web pueda comunicarse con db.

Cuando avancemos con las clases y algunas nociones de redes vamos a profundizar sobre algunas potencialidades de Docker y su uso.

https://docs.docker.com/